

Motivation

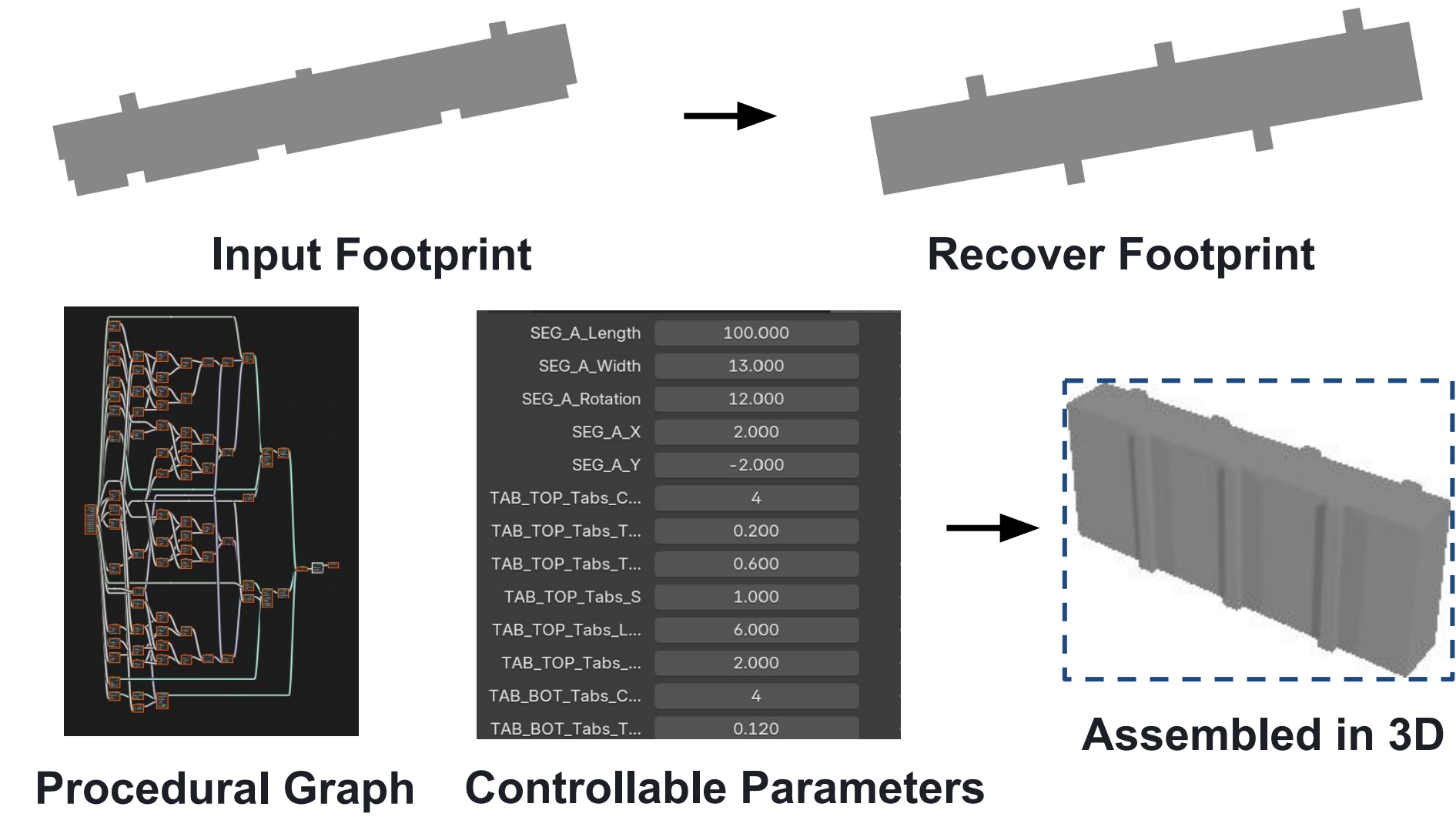
What simulation demands

1. **Watertight.** Simulations need a closed surface
 2. **Controllable.** A design study needs parameters to vary.
 3. **Stable topology.** Those edits must leave the building connected.
- A procedural graph satisfied above conditions.

Task

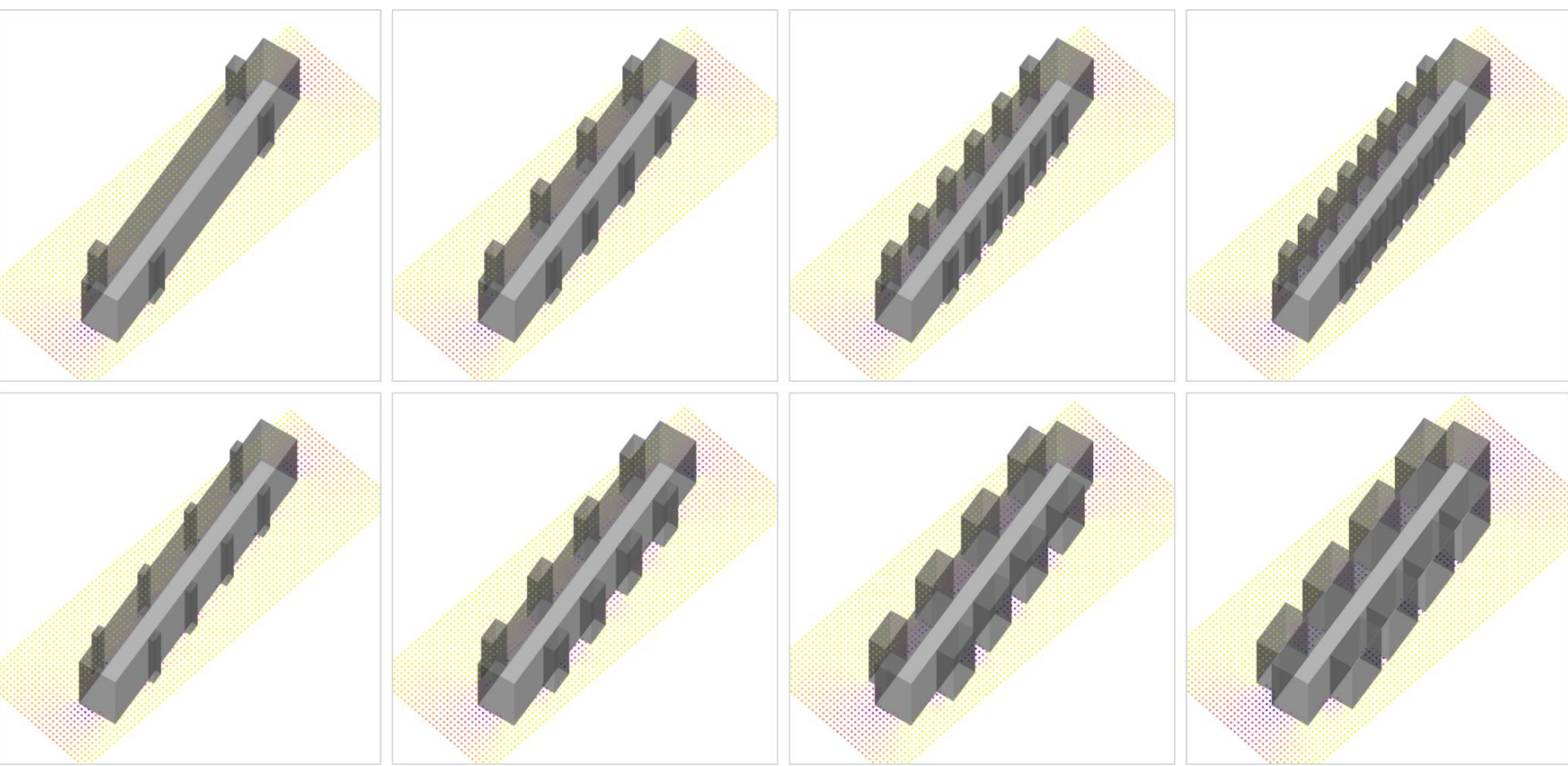
Given a single building footprint image, recover a procedural graph that reproduces it and stays editable. No ground-truth graphs exist, so the problem is addressed zero-shot.

1 RECOVER

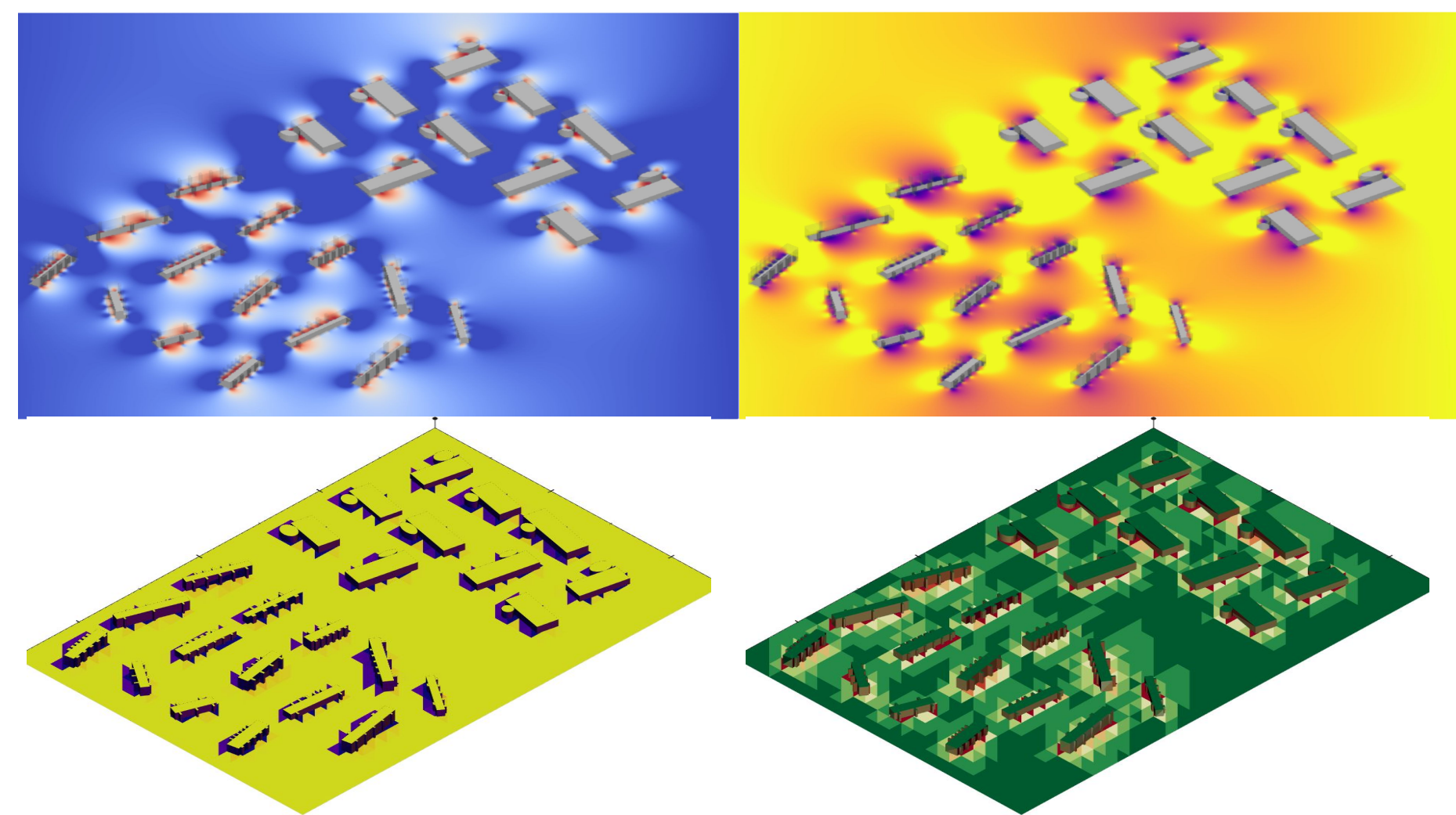


2 EDIT

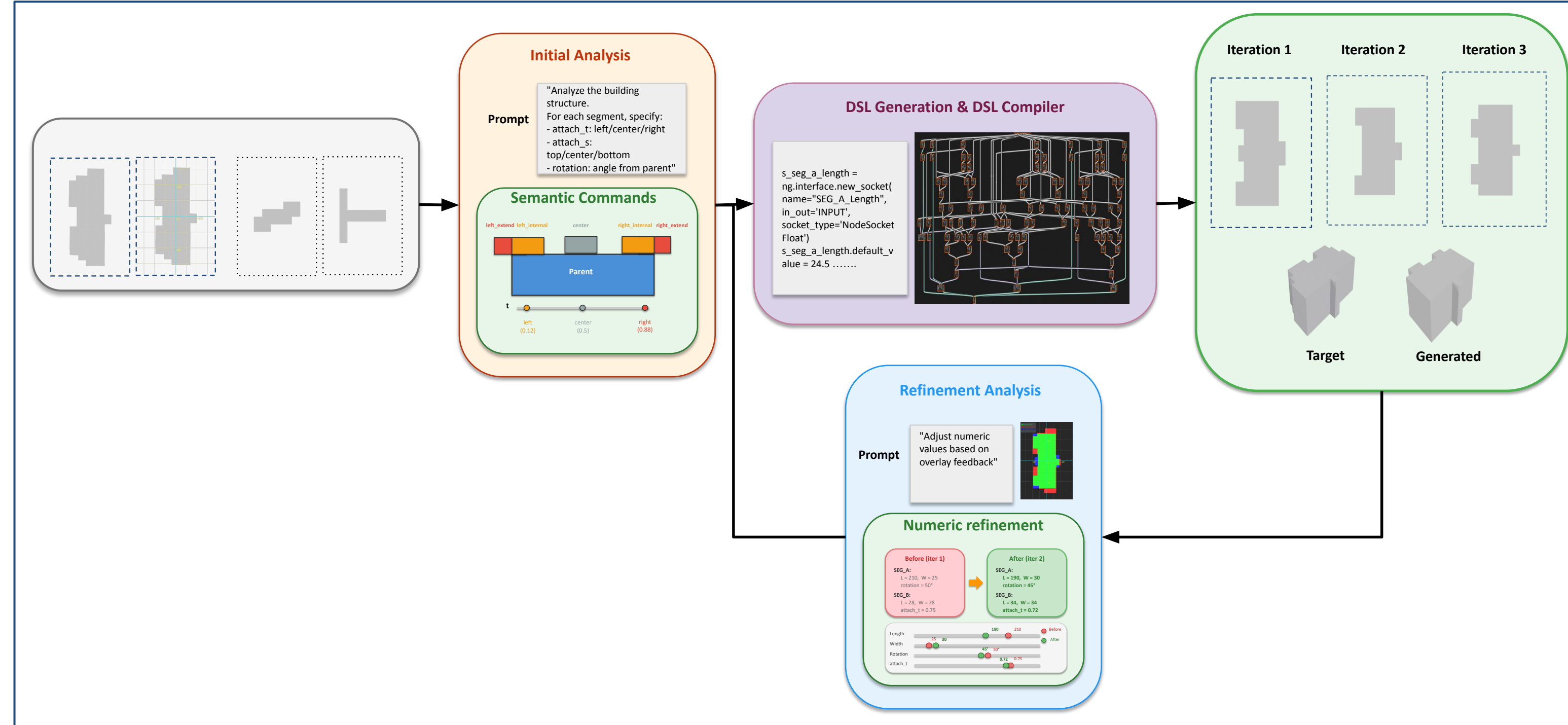
Parameter Variation



3 SIMULATE

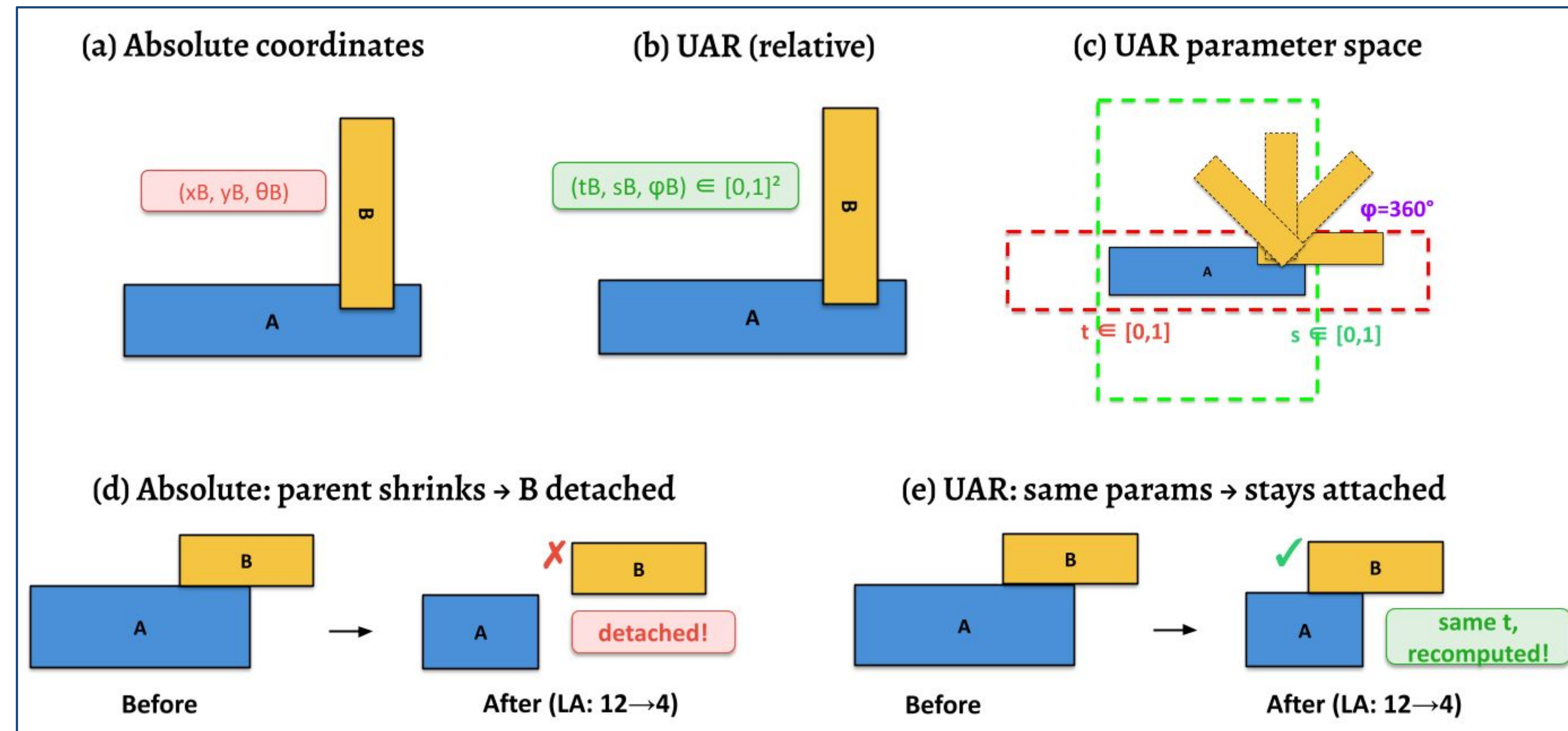


Method



Structural Analysis decomposes the footprint into segments with a semantic keyword. DSL Synthesis produces executable code. Overlay feedback compares the render against the target. Categorical commands are easier for a VLM to generate than continuous coordinates.

Universal Anchor Representation



Absolute coordinates require unbounded (x, y, θ) per segment. UAR describes each child with bounded $(t, s) \in [0,1]^2$ and a rotation ϕ relative to its parent. When the parent shrinks, absolute coordinates leave the child detached, whereas the same (t, s) repositions it.

DSL VS Original Code

```
G = begin_scene(name="Building")
emit_nodes_setup(G)

SEG_A = add_segment(G, "SEG_A", length=92.0, width=11.0,
x=1.0, y=0.0, rotation="right_10")

SEG_B = add_segment(G, "SEG_B", parent=SEG_A, length=18.0, width=13.0,
rotation="right_10", attach_t="left_align",
attach_s="center")

SEG_C = add_segment(G, "SEG_C", parent=SEG_A, length=22.0, width=16.0,
rotation="right_10", attach_t="right_align",
attach_s="center")

SEG_D = add_segment(G, "SEG_D", parent=SEG_A, length=4.0, width=3.0,
rotation="right_10", attach_t=0.38,
attach_s="above_flush")

... SEG_E, SEG_F, SEG_G

geom = join(G, [SEG_A.geom, ..., SEG_G.geom])
geom = merge_by_distance(G, geom)
finish(G, geom)
```

VS

```
mesh = bpy.data.meshes.new("Building_Mesh")
obj = bpy.data.objects.new("Building", mesh)
bpy.context.collection.objects.link(obj)

nod = obj.modifiers.new(name="GeometryNodes", type="NODES")
ng = bpy.data.node_groups.new("FootprintNodes", "GeometryNodeTree")
nod.node_group = ng

ng.interface.clear()
ng.interface.new_socket(name="Geometry", in_out="OUTPUT",
socket_type="NodeSocketGeometry")

nodes, links = ng.nodes, ng.links
nodes.clear()

gin = nodes.new("NodeGroupInput")
gout = nodes.new("NodeGroupOutput")

h = NodeHelpers(nodes, links)

def nmath(nodes, links, op, x=None, y=None, dv1=None, dv2=None):
m = nodes.new("ShaderNodeMath"): m.operation = op
if dv1 is not None: m.inputs[0].default_value = float(dv1)
if dv2 is not None: m.inputs[1].default_value = float(dv2)
if x is not None: links.new(x, m.inputs[0])
if y is not None: links.new(y, m.inputs[1])
return m

def vcombine(nodes, links, x=None, y=None, z=None,
x_default=0.0, y_default=0.0, z_default=0.0):
v = nodes.new("ShaderNodeCombineXYZ")
if x is not None: links.new(x, v.inputs[0])
else: v.inputs[0].default_value = float(x_default)
if y is not None: links.new(y, v.inputs[1])
else: v.inputs[1].default_value = float(y_default)
if z is not None: links.new(z, v.inputs[2])
else: v.inputs[2].default_value = float(z_default)
return v

def rotate_vectors(nodes, links, x_var, y_var, rad_var):
new = nodes.new("NodeSocketGeometry")
```

Quantitative Comparison

+32.1

F1 over the
baseline

100%

connected, every
building

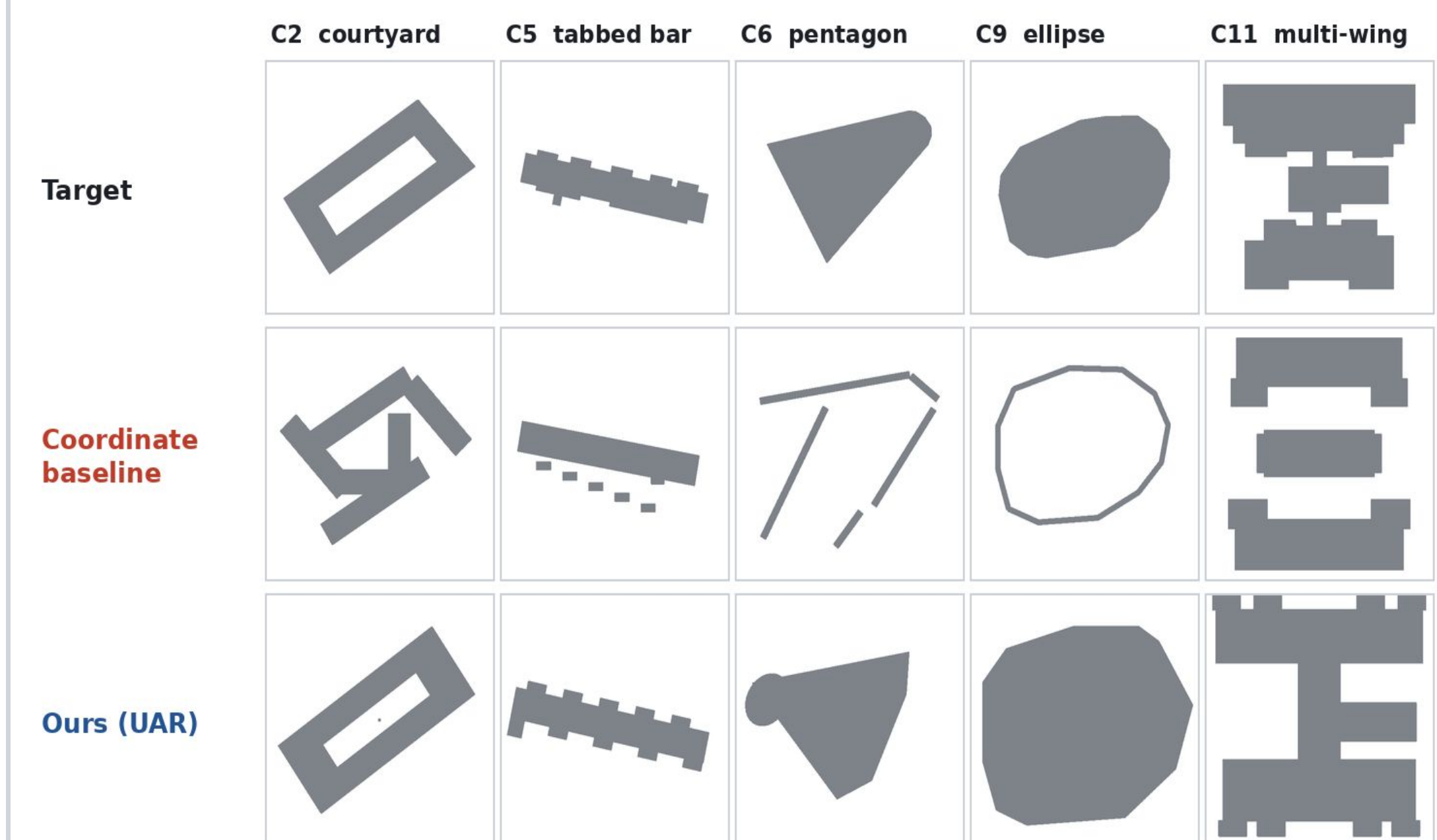
0.34×

output tokens of
the baseline

	F1	Precision	Recall	LCR	Compile
Coordinate baseline	43.1	41.3	46.0	52	57%
Ours	75.2	72.7	80.3	100	100%

After three refinement iterations we reach 78.6 F1 against 68.3, still with 100 LCR and 100% compilation.

Visual Comparison



Ablation

1. Generalization

City	F1	City	F1
Paris	81.0	Chicago	85.3
Barcelona	76.0	Manhattan	82.6
London	88.6	Tokyo	77.2
Average	81.8	LCR, every city	100

Randomly sampled under same primitives without city-specific tuning.

2. Average tokens per building

Method	Input	Output	Output cost
Coordinate baseline	6,382	2,866	1.00×
Ours (DSL)	6,391	981	0.34×

Conclusion: Ready2Sim recovers an editable, simulation-ready parametric graph from a single image. Extending the primitive library and automated topology discovery would widen its reach.